



**ESCUELA
POLITÉCNICA
NACIONAL**
**ESCUELA DE FORMACIÓN
DE TECNÓLOGOS**
TECNOLOGÍA SUPERIOR EN
DESARROLLO DE SOFTWARE



FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL
PROYECTO DEL SEGUNDO BIMESTRE

EDUFEEL IA

Sistema inteligente para el análisis de opiniones estudiantiles



ASIGNATURA	Fundamentos de Inteligencia Artificial
PROFESORA	Ing. Vanessa Guevara MSc.
PERÍODO ACADÉMICO	2026-A
ESTUDIANTE(S)	Ariel Paz Carlos Serrano
REPOSITORIO	https://github.com/arifufino/ProyectoIA

Quito, Ecuador - 20 de julio de 2026

RESUMEN

EduFeel IA es una aplicación web educativa que automatiza la clasificación inicial de opiniones estudiantiles escritas en español. El sistema recibe comentarios abiertos y los clasifica en tres categorías: positivo, neutral o negativo. La solución integra un flujo completo de inteligencia artificial: generación y validación de datos, limpieza de texto, representación TF-IDF, entrenamiento configurable con Regresión Logística o Naive Bayes Multinomial, evaluación mediante métricas, exposición del modelo a través de una API REST y consumo desde una interfaz gráfica.

El backend fue desarrollado con FastAPI, la interfaz con Streamlit y la persistencia del historial con SQLite. Los modelos y sus metadatos se almacenan mediante Joblib. El dataset demostrativo contiene 150 comentarios balanceados, con 50 ejemplos por clase. En una reproducción local del código, la división estratificada de 80% para entrenamiento y 20% para prueba produjo 120 y 30 registros, respectivamente. Ambos algoritmos alcanzaron 100% en accuracy, precisión, recall y F1 dentro del conjunto sintético; sin embargo, estos resultados se interpretan como validación funcional y no como evidencia de desempeño real, debido a la homogeneidad y al tamaño limitado del dataset.

La aplicación cumple con los requisitos esenciales del proyecto: entrenar el modelo, configurar parámetros, mostrar métricas, recibir nuevos casos, exponer una API e incorporar nuevos archivos CSV para reentrenamiento acumulativo. El informe documenta la motivación, el estado del arte, la arquitectura, el diseño del modelo, los resultados, las limitaciones, el manual de usuario y el manual técnico.

Palabras clave: análisis de sentimientos, procesamiento de lenguaje natural, TF-IDF, Regresión Logística, Naive Bayes, FastAPI, Streamlit.

ABSTRACT

EduFeel IA is an educational web application designed to support the initial classification of Spanish-language student feedback. It labels open-ended comments as positive, neutral, or negative and implements an end-to-end artificial intelligence workflow: data generation and validation, text cleaning, TF-IDF representation, configurable training with Logistic Regression or Multinomial Naive Bayes, metric-based evaluation, REST API deployment, and a graphical web interface.

The backend uses FastAPI, Streamlit, SQLite, and Joblib. A local reproduction obtained perfect metrics on a stratified synthetic test split; these results validate the prototype's technical integration but must not be generalized without a larger, diverse, ethically collected, and independently labeled dataset.

ÍNDICE GENERAL

Sección	Contenido
1. Introducción	Contexto, problema, motivación, alcance y restricciones.
2. Objetivos	Objetivo general y objetivos específicos.
3. Estado del arte y fundamentos	Análisis de sentimientos educativo, TF-IDF y clasificadores.
4. Metodología de desarrollo	Fases, tecnologías y trazabilidad con los requisitos.
5. Diseño y arquitectura del sistema	Componentes, estructura del repositorio y flujo de datos.
6. Datos y preprocesamiento	Dataset, limpieza, validaciones e incorporación incremental.
7. Modelo de inteligencia artificial	Entrenamiento, parámetros, métricas y persistencia.
8. Backend, API y persistencia	Endpoints, contratos, manejo de errores y SQLite.
9. Interfaz gráfica	Secciones y capturas del sistema.
10. Pruebas, resultados y discusión	Resultados reproducidos, pruebas y análisis crítico.
11. Manual de usuario	Instalación básica, uso y solución de problemas.
12. Manual técnico	Configuración, estructura, API, pruebas y mantenimiento.
13. Seguridad, ética y limitaciones	Riesgos, privacidad y uso responsable.
14. Conclusiones	Síntesis de resultados.
15. Recomendaciones y trabajos futuros	Mejoras técnicas y académicas.
Referencias	Fuentes académicas y documentación oficial.
Anexos	Trazabilidad, ejemplos de API, video y evidencia de participación.

1. INTRODUCCIÓN

1.1 Contexto

Las instituciones educativas recopilan opiniones estudiantiles para conocer la experiencia de aprendizaje, detectar dificultades y orientar mejoras. Cuando los comentarios son abiertos, su revisión manual puede ser lenta, inconsistente y difícil de escalar. El análisis de sentimientos permite transformar texto no estructurado en categorías que facilitan una primera lectura cuantitativa, sin reemplazar la interpretación humana ni el análisis pedagógico.

Diversos estudios han aplicado técnicas de aprendizaje automático al feedback estudiantil. La literatura reporta el uso de TF-IDF, Naive Bayes, máquinas de soporte vectorial, regresión logística y modelos profundos para clasificar polaridad y extraer información útil de comentarios académicos [3]-[5]. EduFeel IA adopta un enfoque clásico, interpretable y apropiado para un prototipo académico: TF-IDF combinado con dos clasificadores supervisados.

1.2 Planteamiento del problema

El problema abordado consiste en clasificar automáticamente comentarios estudiantiles en español como positivos, neutrales o negativos. La aplicación debe permitir que un usuario no especializado introduzca un comentario, obtenga una clasificación con probabilidades, consulte el historial y visualice las métricas del modelo. Además, el sistema debe permitir entrenar y reentrenar el modelo con parámetros configurables y nuevos datos etiquetados.

1.3 Motivación y justificación

La motivación principal es demostrar un ciclo completo de desarrollo de inteligencia artificial dentro de una aplicación funcional. El proyecto no se limita a un cuaderno de entrenamiento: integra la preparación de datos, el modelo, una API, una interfaz web, persistencia y pruebas automatizadas. Esta integración responde directamente a las indicaciones académicas, que exigen backend, frontend, entrenamiento, configuración, métricas y recepción de nuevos casos [1].

La elección de métodos tradicionales reduce el costo computacional, facilita la ejecución local y permite explicar con claridad cada etapa durante la defensa. TF-IDF convierte los comentarios en características numéricas; la Regresión Logística y Naive Bayes ofrecen líneas base sólidas para clasificación de texto y pueden entrenarse rápidamente en equipos de uso académico [6]-[8].

1.4 Alcance

El alcance funcional de EduFeel IA comprende la clasificación de comentarios cortos en español, el entrenamiento con un CSV etiquetado, la incorporación acumulativa de nuevos comentarios, la consulta de métricas, el almacenamiento del historial y la exposición de servicios REST. El sistema se ejecuta de forma local y está orientado a fines demostrativos y educativos.

No se incluyen autenticación, gestión de usuarios, despliegue productivo, anonimización automática, análisis por temas, detección de sarcasmo ni toma de decisiones institucionales. Tampoco se pretende que la clasificación sustituya la revisión humana de comentarios sensibles o ambiguos.

2. OBJETIVOS

2.1 Objetivo general

Desarrollar una aplicación web funcional en Python que aplique un modelo de inteligencia artificial para clasificar opiniones estudiantiles en español como positivas, neutrales o negativas, incorporando entrenamiento configurable, métricas de evaluación, API REST, interfaz gráfica e historial persistente.

2.2 Objetivos específicos

- Construir y validar un dataset etiquetado con las categorías positivo, neutral y negativo.
- Implementar un proceso de limpieza de texto que conserve caracteres propios del idioma español y elimine entradas inválidas o duplicadas.
- Representar los comentarios mediante TF-IDF y comparar Regresión Logística con Naive Bayes Multinomial.
- Permitir la configuración del tamaño de prueba, número máximo de características, rango de n-gramas, iteraciones y parámetro alpha.
- Calcular accuracy, precisión, recall, F1-score, reporte por clase y matriz de confusión.
- Exponer el entrenamiento, la predicción, las métricas, la información del modelo y el historial a través de FastAPI.
- Desarrollar una interfaz gráfica en Streamlit para operar el sistema sin utilizar comandos técnicos.
- Persistir el modelo mediante Joblib y el historial de predicciones mediante SQLite.
- Implementar pruebas automatizadas y documentar el sistema mediante manual técnico, manual de usuario e informe final.

3. ESTADO DEL ARTE Y FUNDAMENTOS

3.1 Análisis de sentimientos en educación

El análisis de sentimientos busca identificar la orientación afectiva expresada en un texto. En el contexto educativo, se utiliza para organizar retroalimentación abierta y reconocer patrones de satisfacción, neutralidad o inconformidad. Investigaciones recientes han evaluado múltiples modelos sobre comentarios estudiantiles y de docentes, empleando métricas como accuracy, precisión, recall y F1-score [3], [5].

Nasim, Rajput y Haider combinaron características TF-IDF y léxicas para analizar feedback estudiantil, mostrando la pertinencia de las técnicas supervisadas en este dominio [4]. Trabajos posteriores han incorporado redes neuronales y representaciones contextuales, pero también señalan que la calidad de las etiquetas, el sesgo y la privacidad son retos centrales. Para un proyecto académico pequeño, los clasificadores clásicos ofrecen una relación adecuada entre rendimiento, velocidad, explicabilidad y facilidad de implementación.

3.2 Representación TF-IDF

Los algoritmos de clasificación no procesan directamente las palabras; requieren una representación numérica. TF-IDF asigna un peso alto a los términos frecuentes en un documento pero poco frecuentes en el conjunto total. De forma simplificada:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log((1 + N) / (1 + \text{DF}(t))) + 1$$

En la implementación se utiliza `TfidfVectorizer`, que aprende el vocabulario y devuelve una matriz documento-término con características TF-IDF [6]. El rango de n-gramas configurable permite considerar palabras individuales y secuencias de dos palabras, lo cual ayuda a capturar expresiones como “poco clara” o “muy bien”.

3.3 Regresión Logística

Aunque su nombre contiene la palabra regresión, `LogisticRegression` se utiliza como clasificador. Estima probabilidades por clase a partir de una combinación lineal de las características. En EduFeel IA se configura con `class_weight='balanced'`, `max_iter` configurable y `random_state=42`. Su principal ventaja es producir probabilidades y funcionar bien con matrices dispersas de texto [7].

3.4 Naive Bayes Multinomial

`MultinomialNB` es un clasificador probabilístico común en tareas de texto. Modela la distribución de características por clase y utiliza el parámetro `alpha` para suavizado. Es rápido, requiere pocos recursos y constituye una línea base útil para comparar con la Regresión Logística [8].

3.5 Arquitectura web para modelos de IA

El proyecto separa la interfaz, la API, el modelo y la persistencia. `FastAPI` genera documentación interactiva basada en `OpenAPI` y facilita la validación de datos mediante modelos tipados [9]. `Streamlit` permite construir interfaces de datos con widgets, tablas y gráficos directamente en Python [10]. `SQLite` aporta una base transaccional embebida, sin servidor y de configuración mínima [11]. Esta combinación es adecuada para un prototipo reproducible que puede ejecutarse en un equipo local.

4. METODOLOGÍA DE DESARROLLO

4.1 Enfoque de trabajo

Se aplicó un proceso incremental orientado a entregables. Cada fase produjo un componente verificable y reutilizable por la siguiente: estructura del repositorio, dataset, preprocesamiento, entrenamiento, API, interfaz, persistencia, pruebas y documentación. Este enfoque reduce la probabilidad de integrar al final componentes incompatibles.

Fase	Resultado principal
Fase 1	Definición del problema y criterios de aceptación.
Fase 2	Generación del dataset balanceado y validación del esquema.
Fase 3	Construcción del pipeline TF-IDF + clasificador.
Fase 4	Cálculo de métricas y persistencia del artefacto.
Fase 5	Desarrollo de API REST y contratos HTTP.
Fase 6	Implementación de interfaz Streamlit e historial SQLite.
Fase 7	Pruebas automáticas, manejo de errores y documentación.

4.2 Tecnologías utilizadas

Tecnología	Uso en el proyecto
Python	Lenguaje principal del backend, frontend y modelo.
Pandas	Lectura, limpieza, combinación y validación de archivos CSV.
Scikit-learn	TF-IDF, división de datos, modelos y métricas.
FastAPI	API REST, validación de solicitudes y documentación OpenAPI.
Streamlit	Interfaz gráfica web con formularios, métricas y tablas.

Tecnología	Uso en el proyecto
SQLite	Persistencia local del historial de predicciones.
Joblib	Serialización del pipeline, configuración y métricas.
Pytest	Pruebas unitarias y de integración.
Git/GitHub	Control de versiones y publicación del proyecto.

4.3 Correspondencia con los requisitos académicos

Requisito	Evidencia	Estado
Aplicación en Python con interfaz gráfica	Frontend Streamlit y backend FastAPI.	Cumplido
Entrenar el modelo	POST /api/train y POST /api/train/upload.	Cumplido
Configurar parámetros	Algoritmo, test_size, max_features, n-gramas, max_iter y alpha.	Cumplido
Mostrar métricas	Accuracy, precisión, recall, F1, reporte y matriz.	Cumplido
Recibir un nuevo caso	POST /api/predict y formulario de análisis.	Cumplido
Repositorio público	Repositorio arifufino/ProyectoIA.	Cumplido
Commits progresivos de integrantes	La vista pública consultada mostró un solo commit; debe verificarse antes de entregar.	Pendiente
Manual técnico y documentación con capturas	Incluidos en este informe.	Cumplido
Video tutorial	Debe pegarse el enlace en el Anexo C.	Pendiente

5. DISEÑO Y ARQUITECTURA DEL SISTEMA

5.1 Arquitectura lógica

La arquitectura sigue una separación por capas. Streamlit actúa como cliente y envía solicitudes HTTP a FastAPI. El backend valida datos, carga o entrena el pipeline, persiste predicciones y devuelve respuestas JSON. La interfaz no accede de forma directa al archivo Joblib ni a SQLite, lo cual centraliza las reglas de negocio y simplifica el mantenimiento.

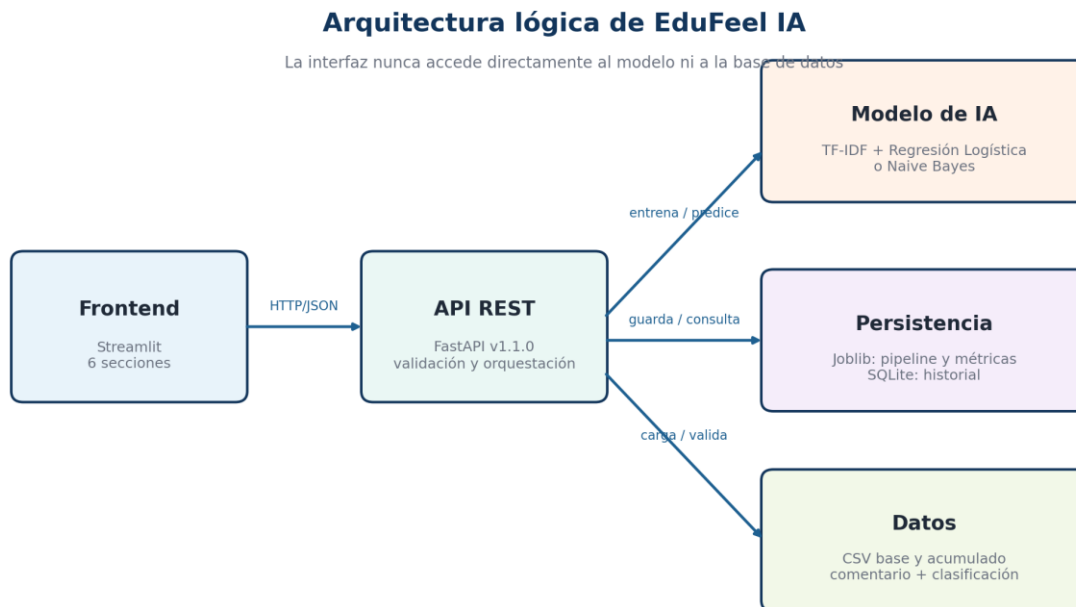


Figura 1. Arquitectura lógica de EduFeel IA.

5.2 Estructura del repositorio

```

ProyectoIA/
├── backend/      # API FastAPI y esquemas Pydantic
├── frontend/     # interfaz Streamlit
├── training/     # limpieza, validación, entrenamiento y predicción
├── data/         # comentarios.csv y dataset_acumulado.csv
├── database/     # repositorio SQLite y archivo generado
├── saved_models/ # artefactos Joblib generados
├── scripts/     # generación reproducible del dataset
├── tests/       # pruebas Pytest
├── docs/        # manuales, informe y guiones
├── README.md
├── PROGRESS.md
└── requirements.txt

```

La estructura modular permite probar cada responsabilidad de forma independiente. Los archivos generados durante la ejecución, como el modelo .joblib y la base .db, se excluyen del control de versiones y se recrean al entrenar o utilizar el sistema.

5.3 Flujo de una predicción

1. El usuario escribe un comentario en la sección Analizar comentario.
2. Streamlit envía un POST a /api/predict con un objeto JSON.
3. FastAPI valida que el texto no esté vacío y carga el artefacto Joblib.
4. La función `clean_text` normaliza el comentario.
5. El pipeline aplica la misma transformación TF-IDF utilizada en entrenamiento.
6. El clasificador calcula probabilidades y selecciona la clase con mayor valor.
7. La predicción se almacena en SQLite y se devuelve a la interfaz.
8. Streamlit presenta la clase, la confianza, las probabilidades y la fecha.

6. DATOS Y PREPROCESAMIENTO

6.1 Dataset demostrativo

El archivo `data/comentarios.csv` se genera mediante un script reproducible. Contiene diez asignaturas y cinco plantillas por categoría. La combinación produce 50 comentarios positivos, 50 neutrales y 50 negativos. El balance evita que una clase domine el entrenamiento y facilita el uso de una partición estratificada.

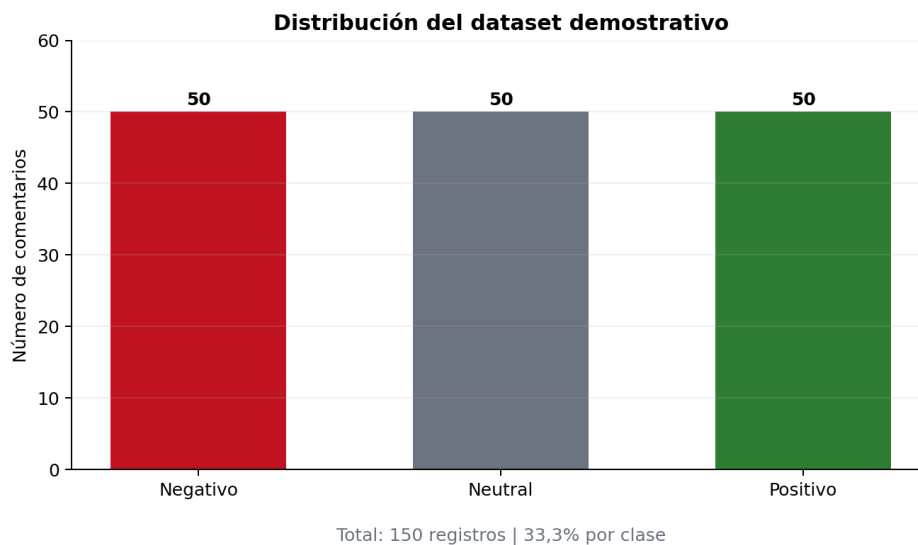


Figura 2. Distribución del dataset demostrativo.

6.2 Esquema requerido

Los archivos CSV deben usar codificación UTF-8 y contener exactamente las columnas mínimas comentario y clasificacion. Las etiquetas aceptadas son positivo, neutral y negativo. Para entrenar un dataset completo se requieren al menos diez registros por clase.

```
comentario,clasificacion
La explicación fue clara y útil.,positivo
La sesión se realizó según lo planificado.,neutral
Las instrucciones fueron confusas.,negativo
```

6.3 Limpieza y validación

Validación	Comportamiento
Conversión a texto	Transforma valores válidos y descarta nulos.
Minúsculas	Reduce variaciones por capitalización.
Caracteres permitidos	Conserva letras españolas, números, espacios y puntuación básica.
Espacios	Agrupar múltiples espacios en uno.
Vacíos	Elimina comentarios y etiquetas vacías.
Duplicados	Conserva la primera aparición de cada comentario.
Categorías	Rechaza etiquetas distintas de positivo, neutral o negativo.
Mínimos	Exige presencia y al menos diez registros por clase para entrenamiento completo.

6.4 Incorporación acumulativa de nuevos datos

La versión actual incluye el endpoint `/api/train/upload` y un cargador de archivos en la interfaz. El archivo nuevo se valida sin exigir que por sí solo contenga diez registros por clase. Luego se combina con el dataset activo, se eliminan duplicados y el conjunto completo se valida antes de guardarse como `data/dataset_acumulado.csv`. Finalmente, el modelo se reentrena con todos los datos acumulados.

7. MODELO DE INTELIGENCIA ARTIFICIAL

7.1 Pipeline de entrenamiento

Flujo de entrenamiento y evaluación



Figura 3. Flujo de entrenamiento y evaluación.

La implementación usa Pipeline de Scikit-learn para encadenar la vectorización y el clasificador. Esto garantiza que la misma transformación aprendida durante el entrenamiento se aplique a cada comentario nuevo, reduciendo errores por diferencias entre el procesamiento de entrenamiento y predicción.

7.2 Parámetros configurables

Parámetro	Predeterminado	Validación	Finalidad
algorithm	logistic_regression	logistic_regression o naive_bayes	Selecciona el clasificador.
test_size	0.20	0.10 a 0.50	Proporción reservada para prueba.
max_features	5000	>= 100	Máximo de términos o n-gramas.
ngram_min	1	1 a 3	Tamaño mínimo de n-grama.
ngram_max	2	1 a 3	Tamaño máximo de n-grama.
max_iter	1000	>= 100	Iteraciones máximas de Regresión Logística.
alpha	1.0	> 0	Suavizado de Naive Bayes.
random_state	42	Fijo en el código	Reproducibilidad de la partición.

7.3 División de entrenamiento y prueba

Se utiliza `train_test_split` con `stratify=data['clasificacion']` y `random_state=42`. La estratificación conserva aproximadamente la misma proporción de clases en ambos subconjuntos [13]. Con 150 registros y `test_size=0.20`, se generan 120 ejemplos de entrenamiento y 30 de prueba: diez por clase en el conjunto de evaluación.

7.4 Métricas de evaluación

Métrica	Interpretación
Accuracy	Proporción total de predicciones correctas.
Precisión	De los elementos predichos como una clase, cuántos pertenecen realmente a ella.
Recall	De los elementos reales de una clase, cuántos fueron recuperados.
F1-score	Media armónica entre precisión y recall.
Matriz de confusión	Distribución de aciertos y errores por clase real y predicha.
Reporte por clase	Precisión, recall, F1 y soporte para cada etiqueta.

7.5 Persistencia del modelo

Joblib guarda un diccionario que contiene el pipeline entrenado, las métricas, la configuración, la fecha UTC, el número de muestras y las clases. De esta forma, la API puede reiniciarse y reutilizar el modelo sin entrenarlo nuevamente. La documentación de Joblib advierte que load se basa en pickle y puede ejecutar código arbitrario; por ello, el sistema solo debe cargar artefactos generados y controlados por el propio proyecto [12].

8. BACKEND, API Y PERSISTENCIA

8.1 API REST

FastAPI centraliza las operaciones y presenta la versión 1.1.0 del servicio. Los esquemas Pydantic validan la longitud del comentario y definen los parámetros de entrenamiento. La documentación interactiva se encuentra en /docs cuando el servidor está activo [9].

Método	Ruta	Propósito
GET	/api/health	Estado del servicio y existencia del modelo.
POST	/api/train	Entrena con el dataset base o acumulado.
POST	/api/train/upload	Incorpora un CSV, elimina duplicados y reentrena.
POST	/api/predict	Clasifica un comentario y lo guarda en el historial.
GET	/api/metrics	Devuelve las métricas del artefacto actual.
GET	/api/history	Devuelve hasta 500 predicciones recientes.
GET	/api/model-info	Configuración, fecha, muestras y clases del modelo.

FastAPI EduFeel IA API 1.1.0

EduFeel IA API
OpenAPI 3.1

GET	/api/health	Estado del servicio y del modelo
POST	/api/train	Entrenar con el dataset activo
POST	/api/train/upload	Agregar CSV y reentrenar
POST	/api/predict	Clasificar y guardar comentario
GET	/api/metrics	Consultar métricas actuales
GET	/api/history	Obtener historial SQLite
GET	/api/model-info	Configuración y fecha del modelo

Figura 4. Vista representativa de la documentación interactiva de la API.

8.2 Manejo de errores

Caso	Respuesta
422	Comentario vacío, parámetros inválidos, CSV incorrecto, categorías no válidas o datos insuficientes.
503	El modelo no ha sido entrenado o no puede cargarse.
Error de conexión	La interfaz informa que FastAPI no está disponible.
Archivo no CSV	El endpoint de carga rechaza extensiones distintas de .csv.

8.3 Persistencia SQLite

Cada predicción almacena identificador, comentario, clasificación, confianza, probabilidades y fecha. SQLite es adecuado para este prototipo porque se ejecuta dentro del proceso y guarda la base en un archivo local, sin necesidad de instalar un servidor independiente [11].

La base sirve como historial de uso, pero no debe considerarse un repositorio institucional seguro. Para datos reales se requerirían autenticación, control de acceso, cifrado, política de retención y anonimización.

9. INTERFAZ GRÁFICA

9.1 Navegación principal

Streamlit presenta seis secciones en la barra lateral: Inicio, Analizar comentario, Entrenar modelo, Métricas, Historial e Información del proyecto. La interfaz consume exclusivamente la API, por lo que las reglas de negocio permanecen en el backend.

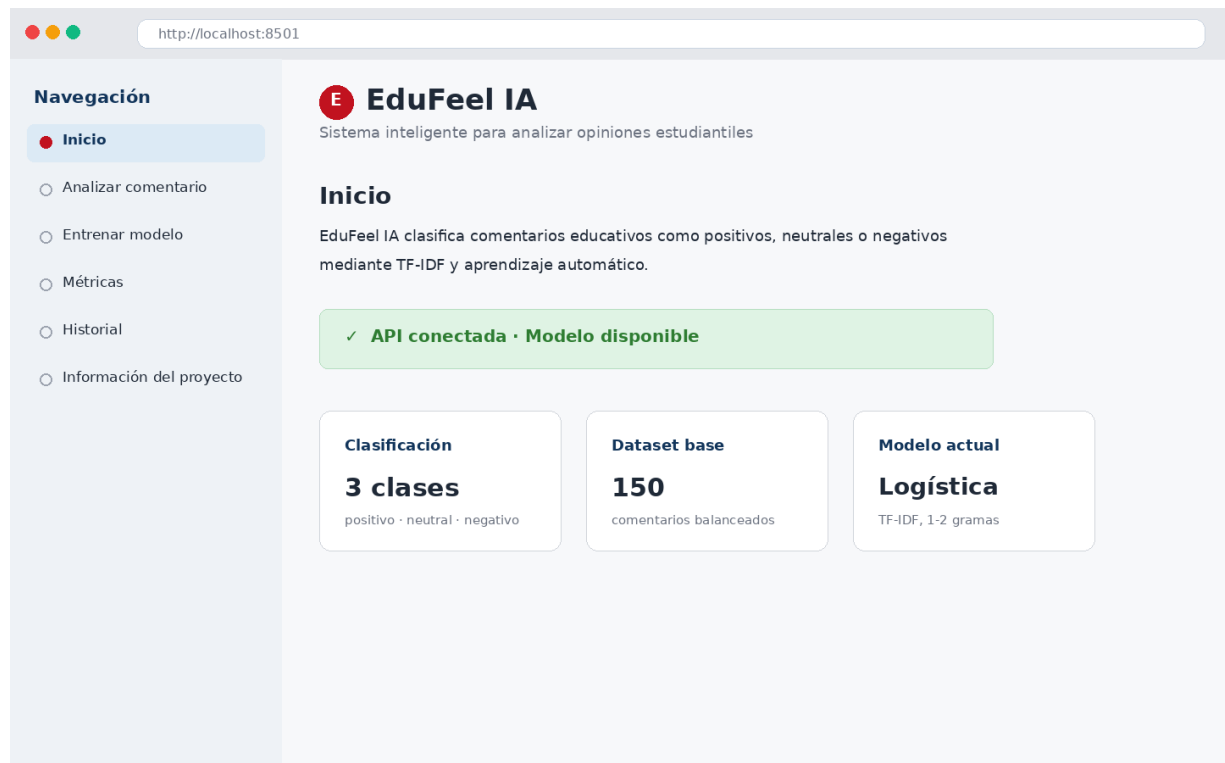


Figura 5. Pantalla de inicio y estado de conexión.

9.2 Análisis de comentarios

El usuario escribe un comentario y presiona Analizar. Si el texto es válido y existe un modelo, la interfaz muestra la clase en mayúsculas, la confianza, un gráfico de probabilidades y la fecha. La confianza representa la probabilidad estimada por el modelo, no una garantía de corrección.



Figura 6. Resultado de clasificación de un comentario positivo.

9.3 Entrenamiento y métricas

La sección Entrenar modelo permite seleccionar el algoritmo y modificar los parámetros. También acepta un CSV nuevo, informa cuántos registros se agregaron y cuántos duplicados se omitieron. La sección Métricas presenta indicadores globales, matriz de confusión y reporte por clase.

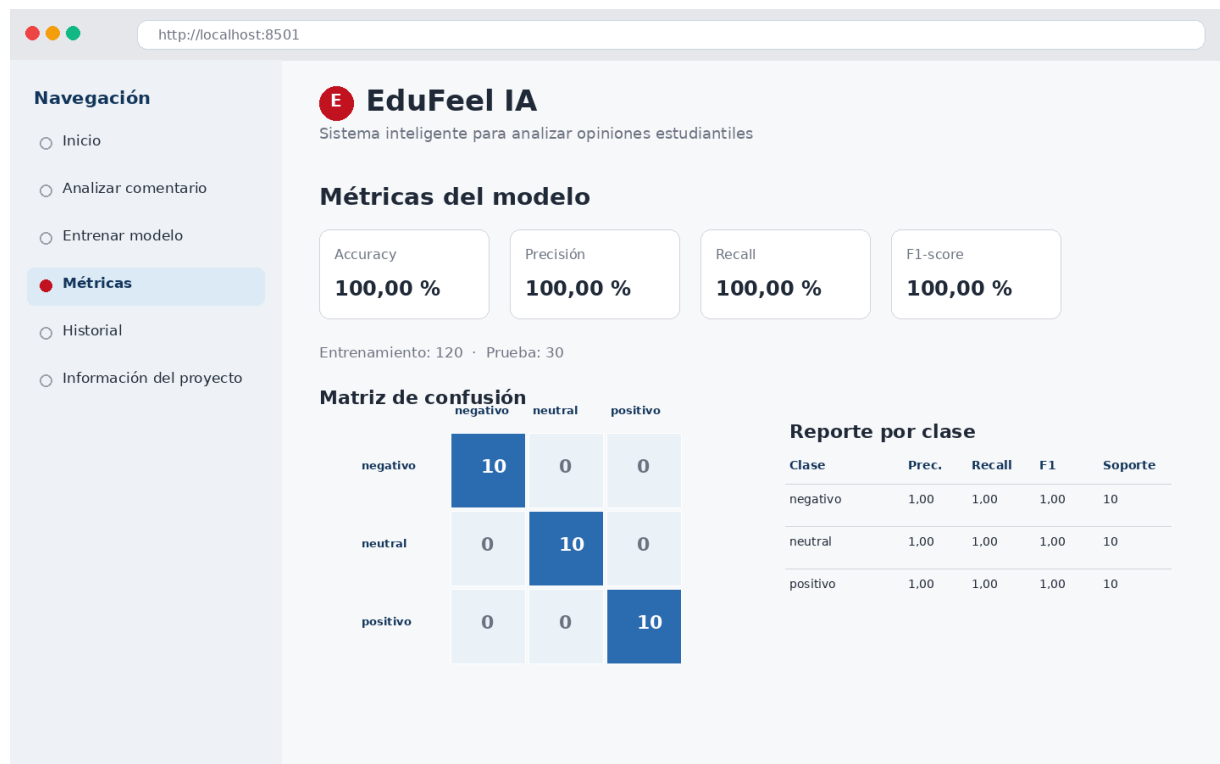


Figura 7. Métricas y matriz de confusión del modelo.

9.4 Historial

El historial consulta SQLite a través de la API y presenta las predicciones más recientes. El usuario puede seleccionar entre 10 y 200 registros desde la interfaz; el endpoint admite hasta 500.

EduFeel IA
Sistema inteligente para analizar opiniones estudiantiles

Historial de predicciones
Cantidad: 50

ID	Comentario	Clasificación	Confianza	Fecha
3	La explicación de física fue poco clara y demasiado ráp...	negativo	81,92 %	20/07/2026 11:15
2	Hoy revisamos el contenido de matemáticas y completamos...	neutral	78,71 %	20/07/2026 11:14
1	La clase de programación fue excelente y aprendí mucho.	positivo	76,08 %	20/07/2026 11:13

Figura 8. Historial de predicciones persistidas.

10. PRUEBAS, RESULTADOS Y DISCUSIÓN

10.1 Verificación reproducida

Para validar el comportamiento descrito en el repositorio, se reprodujo localmente el proceso de generación del dataset y entrenamiento usando el código publicado. Se entrenaron ambos algoritmos con los parámetros predeterminados. La partición estratificada produjo 120 registros de entrenamiento y 30 de prueba.

Modelo	Accuracy	Precisión	Recall	F1	Train/Test
Regresión Logística	1,0000	1,0000	1,0000	1,0000	120 / 30
Naive Bayes Multinomial	1,0000	1,0000	1,0000	1,0000	120 / 30

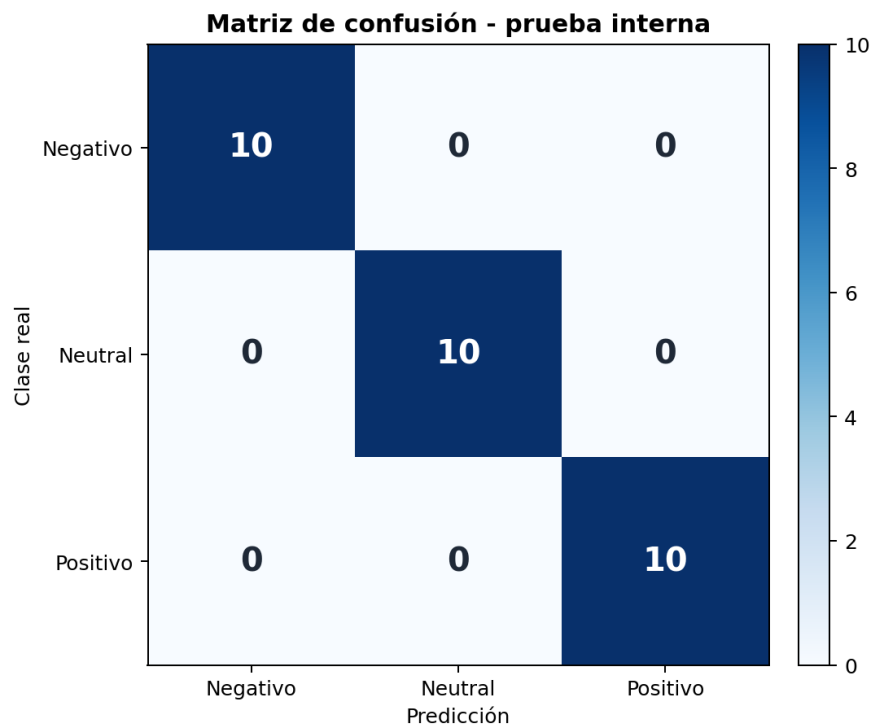


Figura 9. Matriz de confusión obtenida con el dataset sintético.

10.2 Casos de aceptación

Comentario	Predicción	Confianza	Resultado
La clase de programación fue excelente y aprendí mucho.	Positivo	76,08%	Correcto
Hoy revisamos el contenido de matemáticas y completamos las actividades.	Neutral	78,71%	Correcto
La explicación de física fue poco clara y demasiado rápida.	Negativo	81,92%	Correcto

10.3 Pruebas automatizadas

El archivo PROGRESS.md informa una verificación final con 15 pruebas aprobadas. La suite cubre limpieza, esquema del dataset, categorías, suficiencia de datos, ambos modelos, predicción, errores, endpoints principales y persistencia SQLite. Entre los escenarios se incluyen modelo inexistente, comentario vacío y CSV con columnas incorrectas.

```
pytest
# Resultado documentado en el repositorio: 15 pruebas aprobadas
```

10.4 Interpretación crítica

El 100% obtenido en la prueba interna confirma que el pipeline distingue correctamente las plantillas presentes en el dataset, pero no demuestra generalización. Los 150 registros se construyen a partir de quince patrones lingüísticos repetidos sobre diez asignaturas. Como el entrenamiento y la prueba comparten estructuras muy similares, el modelo puede aprender palabras o frases específicas en lugar de la variedad real del lenguaje estudiantil.

En pruebas exploratorias con redacciones distintas a las plantillas, comentarios mixtos o expresiones ambiguas, la confianza disminuye y pueden ocurrir clasificaciones erróneas. Por esta razón, el sistema debe presentarse como prototipo funcional. La siguiente fase requiere datos reales anonimizados, etiquetado consistente, validación cruzada y un conjunto de prueba independiente que no comparta plantillas de generación.

11. MANUAL DE USUARIO

11.1 Requisitos previos

El usuario necesita un equipo con Python, conexión local entre los dos servicios y un navegador web. La instalación técnica se detalla en el capítulo 12. Para usar la aplicación diariamente, deben permanecer activas dos terminales: una para FastAPI y otra para Streamlit.

11.2 Inicio del sistema

9. Abrir PowerShell en la carpeta raíz del proyecto.
10. Activar el entorno virtual.
11. En la primera terminal ejecutar `uvicorn backend.main:app --reload`.
12. En una segunda terminal ejecutar `streamlit run frontend/app.py`.
13. Abrir `http://localhost:8501` en el navegador.
14. Comprobar en Inicio que aparezca “API conectada”.

11.3 Entrenar el modelo

15. Seleccionar Entrenar modelo en la barra lateral.
16. Elegir Regresión logística o Naive Bayes.
17. Mantener los parámetros predeterminados para una primera ejecución.
18. Presionar Entrenar con datos acumulados.
19. Esperar el mensaje de confirmación y revisar las métricas.

11.4 Agregar nuevos datos

20. Preparar un CSV UTF-8 con las columnas comentario y clasificacion.
21. Verificar que las etiquetas sean positivo, neutral o negativo.
22. Seleccionar el archivo en Dataset nuevo (CSV).
23. Presionar Agregar datos y entrenar.
24. Revisar el total acumulado, registros nuevos y duplicados omitidos.

11.5 Analizar un comentario

25. Ir a Analizar comentario.
26. Escribir una opinión completa y evitar dejar el campo vacío.
27. Presionar Analizar.
28. Interpretar la clase y las probabilidades como una sugerencia del modelo.
29. Revisar manualmente los casos de baja confianza o contenido mixto.

11.6 Consultar métricas e historial

La sección Métricas muestra la evaluación del último entrenamiento. Historial presenta las predicciones guardadas en SQLite. La sección Información del proyecto resume el modelo actual, la fecha de entrenamiento, el tamaño del dataset y la configuración.

11.7 Problemas frecuentes

Mensaje o problema	Solución
La API no está disponible	Ejecutar <code>uvicorn backend.main:app --reload</code> y confirmar el puerto 8000.
El modelo aún no ha sido entrenado	Entrenar desde la interfaz o <code>POST /api/train</code> .
Faltan columnas obligatorias	Renombrar las columnas a comentario y clasificacion.
Categorías incorrectas	Usar únicamente positivo, neutral o negativo.
Dataset insuficiente	Acumular al menos diez registros por clase antes del entrenamiento completo.
Puerto ocupado	Cerrar el proceso anterior o iniciar el servicio en otro puerto y ajustar <code>EDUFEEL_API_URL</code> .

12. MANUAL TÉCNICO

12.1 Instalación

```
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

En Linux o macOS, la activación se realiza con `source .venv/bin/activate`. La instalación debe ejecutarse desde la carpeta raíz para que Python resuelva correctamente los paquetes backend, training y database.

12.2 Ejecución

```
# Terminal 1 - API
uvicorn backend.main:app --reload

# Terminal 2 - interfaz
streamlit run frontend/app.py
```

Direcciones predeterminadas: API <http://localhost:8000>, documentación <http://localhost:8000/docs> e interfaz <http://localhost:8501>.

12.3 Variables de entorno

Variable	Predeterminado	Uso
----------	----------------	-----

Variable	Predeterminado	Uso
EDUFEEL_MODEL_PATH	saved_models/sentiment_model.joblib	Ruta del artefacto Joblib.
EDUFEEL_DB_PATH	database/edufeel.db	Ruta de la base SQLite.
EDUFEEL_DATASET_PATH	data/comentarios.csv	Dataset base.
EDUFEEL_ACCUMULATED_DATASET_PATH	data/dataset_acumulado.csv	Dataset acumulado.
EDUFEEL_API_URL	http://localhost:8000	URL consumida por Streamlit.

12.4 Contratos de API

Ejemplo de entrenamiento con Regresión Logística:

```
POST /api/train
Content-Type: application/json
```

```
{
  "algorithm": "logistic_regression",
  "test_size": 0.2,
  "max_features": 5000,
  "ngram_min": 1,
  "ngram_max": 2,
  "max_iter": 1000,
  "alpha": 1.0
}
```

Ejemplo de predicción:

```
POST /api/predict
Content-Type: application/json
```

```
{
  "comentario": "La explicación fue clara y los ejercicios fueron útiles."
}
```

12.5 Formato del artefacto

Clave	Contenido
pipeline	TfidfVectorizer y clasificador entrenado.
metrics	Métricas, reporte, matriz y tamaños de partición.
config	Parámetros usados en entrenamiento.
trained_at	Fecha y hora UTC del entrenamiento.
dataset_samples	Número total de comentarios.
classes	Clases aprendidas por el modelo.

12.6 Pruebas

```
pytest
pytest -q
pytest tests/test_model.py
pytest tests/test_api.py
```

Las pruebas deben ejecutarse desde la raíz del repositorio. Para una entrega reproducible, conviene guardar una captura o archivo de salida con la fecha, versión de Python y número de pruebas aprobadas.

12.7 Mantenimiento del dataset

- Crear una copia de seguridad de data/dataset_acumulado.csv antes de realizar cambios masivos.
- Evitar editar manualmente el archivo mientras la aplicación está entrenando.
- Revisar duplicados semánticos, no solo duplicados exactos.
- Mantener equilibrio entre clases o evaluar técnicas apropiadas para desbalance.
- Separar un conjunto de prueba real que nunca se utilice para entrenar.
- Registrar la fuente, fecha, consentimiento y criterio de etiquetado de cada lote.

13. SEGURIDAD, ÉTICA Y LIMITACIONES

13.1 Privacidad y uso responsable

Los comentarios educativos pueden incluir información personal, conflictos, acusaciones o datos sensibles. Antes de utilizar datos reales se debe obtener autorización, eliminar identificadores y limitar el acceso. El resultado del modelo no debe emplearse para sancionar automáticamente a estudiantes o docentes. Debe funcionar como apoyo para priorizar revisión humana.

13.2 Riesgos técnicos

Riesgo	Impacto	Mitigación
Dataset sintético	Métricas optimistas y vocabulario limitado.	Recolectar datos reales anonimizados y crear prueba independiente.
CORS abierto	La API acepta orígenes * en desarrollo.	Restringir orígenes en despliegue.
Sin autenticación	Cualquier usuario con acceso puede entrenar o consultar historial.	Agregar roles y autenticación.
Joblib/Pickle	Cargar artefactos no confiables puede ejecutar código.	Aceptar solo archivos generados internamente.
SQLite local	No cubre alta concurrencia ni controles institucionales.	Migrar a PostgreSQL u otra base administrada.
Baja explicabilidad	Solo se muestran probabilidades.	Agregar términos influyentes y revisión de incertidumbre.
Deriva del lenguaje	El vocabulario y los temas cambian.	Monitorear desempeño y reentrenar con datos recientes.

13.3 Limitaciones funcionales

- No identifica tema, emoción específica ni destinatario del comentario.
- No comprende adecuadamente ironía, sarcasmo, negaciones complejas o textos mixtos.
- La confianza del modelo puede estar mal calibrada.
- La limpieza no corrige ortografía ni normaliza variantes lingüísticas.
- El modelo no considera contexto histórico del estudiante o de la asignatura.
- El sistema local no incluye copias de seguridad automáticas ni auditoría de usuarios.

14. CONCLUSIONES

30. EduFeel IA demuestra de forma funcional el ciclo completo de una aplicación de inteligencia artificial: datos, preprocesamiento, entrenamiento, evaluación, persistencia, API e interfaz gráfica.
31. La separación entre Streamlit, FastAPI, el pipeline de Scikit-learn y SQLite produce una arquitectura comprensible y modular, apropiada para una defensa académica y para futuras ampliaciones.
32. La aplicación cumple los requisitos técnicos principales del proyecto, incluyendo entrenamiento configurable, visualización de métricas, recepción de nuevos casos e incorporación acumulativa de archivos CSV.

33. Los resultados perfectos del conjunto de prueba sintético validan la consistencia del prototipo, pero no representan el rendimiento esperado en comentarios reales. La calidad y diversidad de datos es el principal factor que debe mejorarse.
34. La solución debe mantenerse como herramienta de apoyo con revisión humana, especialmente por los riesgos de privacidad, sesgo y clasificación incorrecta en contextos educativos.

15. RECOMENDACIONES Y TRABAJOS FUTUROS

- Recolectar comentarios reales anonimizados de diferentes asignaturas, semestres, estilos docentes y formas de expresión.
- Definir una guía de etiquetado y medir acuerdo entre anotadores para reducir subjetividad.
- Crear un conjunto de prueba externo, sin plantillas compartidas con el entrenamiento.
- Aplicar validación cruzada estratificada y reportar promedio y desviación estándar.
- Comparar modelos adicionales, como SVM lineal, Complement Naive Bayes y modelos preentrenados en español.
- Calibrar probabilidades y establecer un umbral para enviar casos inciertos a revisión humana.
- Agregar análisis temático, filtros por periodo, exportación de resultados y paneles estadísticos.
- Implementar autenticación, roles, control de acceso, anonimización y política de retención de datos.
- Restringir CORS y desplegar backend, frontend y base de datos con configuración segura.
- Completar antes de la entrega el enlace del video, los nombres de integrantes y la evidencia de commits progresivos.

REFERENCIAS

- [1] Escuela Politécnica Nacional, Escuela de Formación de Tecnólogos, “Proyecto segundo bimestre - Fundamentos de Inteligencia Artificial”, período 2026-A, 2026.
- [2] A. Paz et al., “EduFeel IA - ProyectoIA”, repositorio público de GitHub, 2026. Disponible en: <https://github.com/arifufino/ProyectoIA> (consulta: 20-jul-2026).
- [3] S. B. Deshpande et al., “Elevating educational insights: sentiment analysis of faculty feedback using advanced machine learning models”, *Advances in Continuous and Discrete Models*, vol. 2025, art. 89, 2025. doi: 10.1186/s13662-025-03933-9.
- [4] Z. Nasim, Q. Rajput y S. Haider, “Sentiment analysis of student feedback using machine learning and lexicon based approaches”, 2017 5th International Conference on Research and Innovation in Information Systems, 2017. doi: 10.1109/ICRIIS.2017.8002475.
- [5] M. Fargues, S. Kadry, I. A. Lawal, S. Yassine y H. T. Rauf, “Automated Analysis of Open-Ended Students’ Feedback Using Sentiment, Emotion, and Cognition Classifications”, *Applied Sciences*, vol. 13, no. 4, art. 2061, 2023. doi: 10.3390/app13042061.
- [6] Scikit-learn, “TfidfVectorizer”, documentación oficial. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (consulta: 20-jul-2026).
- [7] Scikit-learn, “LogisticRegression”, documentación oficial. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (consulta: 20-jul-2026).
- [8] Scikit-learn, “MultinomialNB”, documentación oficial. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html (consulta: 20-jul-2026).
- [9] FastAPI, “First Steps - Automatic interactive API documentation”, documentación oficial. Disponible en: <https://fastapi.tiangolo.com/tutorial/first-steps/> (consulta: 20-jul-2026).
- [10] Streamlit, “Streamlit documentation”, documentación oficial. Disponible en: <https://docs.streamlit.io/> (consulta: 20-jul-2026).

[11] SQLite, “About SQLite”, documentación oficial. Disponible en: <https://sqlite.org/about.html> (consulta: 20-jul-2026).

[12] Joblib, “Persistence”, documentación oficial. Disponible en: <https://joblib.readthedocs.io/en/stable/persistence.html> (consulta: 20-jul-2026).

[13] Scikit-learn, “train_test_split”, documentación oficial. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html (consulta: 20-jul-2026).

ANEXO A. MATRIZ DE TRAZABILIDAD

ID	Requisito	Implementación	Evidencia
RF-01	Entrenar modelo desde la interfaz.	Entrenar modelo; /api/train.	Prueba de API y captura de métricas.
RF-02	Configurar parámetros.	Controles Streamlit y TrainRequest.	Validación de rangos y entrenamiento.
RF-03	Mostrar métricas.	/api/metrics y show_metrics.	Accuracy, precisión, recall, F1, matriz.
RF-04	Recibir un nuevo comentario.	/api/predict.	Casos positivo, neutral y negativo.
RF-05	Guardar historial.	SQLite repository.	/api/history e interfaz Historial.
RF-06	Cargar nuevos datos.	/api/train/upload.	Validación CSV, acumulación y reentrenamiento.
RNF-01	Manejo de errores.	HTTP 422/503 y mensajes Streamlit.	Pruebas de datos inválidos y modelo ausente.
RNF-02	Reproducibilidad.	random_state=42 y script de dataset.	Mismos tamaños de partición.

ANEXO B. EJEMPLOS DE RESPUESTAS

Respuesta resumida de una predicción:

```
{
  "clasificacion": "positivo",
  "confianza": 0.7608,
  "probabilidades": {
    "negativo": 0.1294,
    "neutral": 0.1098,
    "positivo": 0.7608
  },
  "fecha": "2026-07-20T...+00:00",
  "id": 1
}
```

Respuesta resumida del entrenamiento:

```
{
  "message": "Modelo entrenado correctamente",
  "metrics": {
    "accuracy": 1.0,
    "precision": 1.0,
    "recall": 1.0,
    "f1_score": 1.0,
    "training_samples": 120,
    "test_samples": 30
  }
}
```

ANEXO C. ENLACES DE ENTREGA

Repositorio	https://github.com/arifufino/ProyectoIA
-------------	---

Video tutorial	https://youtu.be/D0Uj2pvkyl8
Presentación	EduFeel Presentacion